

Backpropagation: A Worked Example

Postgraduate Diploma in Data Science

University of Witwatersrand, South Africa

Derrick Nyongesa

May 2026

Contents

1	Introduction	2
2	Problem Setup	3
2.1	Input and Target	3
2.2	Weights and Biases	3
3	Forward Propagation	3
3.1	Hidden Layer	3
3.2	Output Layer	3
4	Loss Computation	4
5	Backpropagation	4
5.1	Error Signal at the Output Layer	4
5.2	Gradients for the Output Layer	4
5.3	Error Signal at the Hidden Layer	4
5.4	Gradients for the Input-to-Hidden Layer	4
6	Weight Updates via Gradient Descent	5
6.1	Update: Output Layer	5
6.2	Update: Input-to-Hidden Layer	5
7	Conclusion	6
	References	7

1. Introduction

A *neural network* is a computational model organized into an input layer, one or more hidden layers, and an output layer. Each node (or artificial neuron) in the network receives a weighted sum of its inputs, adds a bias term, and passes the result through an *activation function* g to produce an *activation value*. Concretely, if $a_1, \dots, a_m$ are the activation values feeding into a node with weights $w_1, \dots, w_m$, bias b , and activation function g , the node's output is

$$a = g\left(\sum_{i=1}^m a_i w_i + b\right).$$

Common choices of activation function include the *sigmoid* $\sigma(z) = \frac{1}{1+e^{-z}}$, the *rectified linear unit* $\text{relu}(z) = \max(0, z)$, and the hyperbolic tangent $\tanh(z)$ [4].

Feedforward propagation. Given a set of input values $a^{(0)}$, the activation values propagate layer-by-layer through the network. If W_k is the weight matrix between layer $k-1$ and layer k , b_k the bias vector at layer k , and g_k its activation function, then

$$a^{(k)} = g_k(a^{(k-1)}W_k + b_k).$$

The output $y = a^{(L)}$ at the final layer L is then compared with the desired target t via a *loss function* $L(y, t)$. A standard choice is the *sum-of-squares loss* [3]

$$L(y, t) = \frac{1}{2} \sum_j (y_j - t_j)^2.$$

Training via gradient descent. Training a neural network amounts to adjusting all weights and biases so that $L(y, t)$ is minimized over the training dataset. The *gradient descent update rule* for a single parameter w is

$$w \leftarrow w - \eta \frac{\partial L}{\partial w},$$

where $\eta > 0$ is the *learning rate*. Because a network may contain thousands of parameters, computing all partial derivatives by hand is impractical. Instead, *backpropagation* exploits the chain rule to propagate error signals backwards through the network layer-by-layer, computing the required partial derivatives efficiently [2, 1].

The remainder of this document presents a complete, numerically explicit example of one forward-pass, loss computation, and backpropagation step on a small network, illustrating exactly how each partial derivative is obtained and how the weights are subsequently updated.

2. Problem Setup

We consider a fully connected neural network with:

- 2 input neurons,
- 1 hidden layer containing 2 neurons,
- 1 output neuron,
- sigmoid activation function σ at every neuron, and
- sum-of-squares loss function.

2.1 Input and Target

$$x = (1, 0), \quad t = 1.$$

2.2 Weights and Biases

Input $\rightarrow$ Hidden layer:

$$W_1 = \begin{bmatrix} 0.5 & -0.5 \\ 0.3 & 0.8 \end{bmatrix}, \quad b_1 = (0.1, -0.1).$$

Hidden layer $\rightarrow$ Output:

$$W_2 = \begin{bmatrix} 0.7 \\ -1.2 \end{bmatrix}, \quad b_2 = 0.05.$$

3. Forward Propagation

3.1 Hidden Layer

Compute the pre-activation vector z_1 :

$$z_1 = xW_1 + b_1 = (1, 0) \begin{bmatrix} 0.5 & -0.5 \\ 0.3 & 0.8 \end{bmatrix} + (0.1, -0.1) = (0.5, -0.5) + (0.1, -0.1) = (0.6, -0.6).$$

Apply the sigmoid activation function:

$$a_1 = (\sigma(0.6), \sigma(-0.6)) \approx (0.6457, 0.3543).$$

3.2 Output Layer

Compute the pre-activation scalar z_2 :

$$z_2 = a_1W_2 + b_2 = (0.6457, 0.3543) \begin{bmatrix} 0.7 \\ -1.2 \end{bmatrix} + 0.05 = 0.4520 - 0.4252 + 0.05 = 0.0768.$$

Apply the sigmoid activation function:

$$y = \sigma(0.0768) \approx 0.5192.$$

4. Loss Computation

$$L(y, t) = \frac{1}{2}(y - t)^2 = \frac{1}{2}(0.5192 - 1)^2 \approx 0.1155.$$

5. Backpropagation

5.1 Error Signal at the Output Layer

Derivative of the loss with respect to the output:

$$\frac{dL}{dy} = y - t = 0.5192 - 1 = -0.4808.$$

Derivative of the sigmoid at z_2 (using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$):

$$\sigma'(z_2) = y(1 - y) = 0.5192 \times 0.4808 \approx 0.2496.$$

Output-layer error signal δ_2 (by the chain rule):

$$\delta_2 = (y - t) \sigma'(z_2) = -0.4808 \times 0.2496 \approx -0.1200.$$

5.2 Gradients for the Output Layer

$$\frac{\partial L}{\partial W_2} = a_1^\top \delta_2 = \begin{bmatrix} 0.6457 \\ 0.3543 \end{bmatrix} (-0.1200) = \begin{bmatrix} -0.0775 \\ -0.0425 \end{bmatrix}.$$

$$\frac{\partial L}{\partial b_2} = \delta_2 = -0.1200.$$

5.3 Error Signal at the Hidden Layer

Backpropagate through W_2 :

$$\delta_2 W_2^\top = -0.1200 \times (0.7, -1.2) = (-0.0840, 0.1440).$$

Derivative of the sigmoid at z_1 :

$$\sigma'(z_1) = (\sigma(0.6)(1 - \sigma(0.6)), \sigma(-0.6)(1 - \sigma(-0.6))) = (0.6457 \times 0.3543, 0.3543 \times 0.6457) \approx (0.2287, 0.2287)$$

Hidden-layer error signal (element-wise product $\circ$):

$$\delta_1 = \delta_2 W_2^\top \circ \sigma'(z_1) = (-0.0840, 0.1440) \circ (0.2287, 0.2287) \approx (-0.0192, 0.0329).$$

5.4 Gradients for the Input-to-Hidden Layer

$$\frac{\partial L}{\partial W_1} = x^\top \delta_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} (-0.0192, 0.0329) = \begin{bmatrix} -0.0192 & 0.0329 \\ 0 & 0 \end{bmatrix}.$$

$$\frac{\partial L}{\partial b_1} = \delta_1 = (-0.0192, 0.0329).$$

6. Weight Updates via Gradient Descent

Let the learning rate be $\eta = 0.1$.

6.1 Update: Output Layer

$$W_2^{\text{new}} = \begin{bmatrix} 0.7 \\ -1.2 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0775 \\ -0.0425 \end{bmatrix} = \begin{bmatrix} 0.7078 \\ -1.1958 \end{bmatrix}.$$

$$b_2^{\text{new}} = 0.05 - 0.1 \times (-0.1200) = 0.0620.$$

6.2 Update: Input-to-Hidden Layer

$$W_1^{\text{new}} = \begin{bmatrix} 0.5 & -0.5 \\ 0.3 & 0.8 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0192 & 0.0329 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.5019 & -0.5033 \\ 0.3000 & 0.8000 \end{bmatrix}.$$

$$b_1^{\text{new}} = (0.1, -0.1) - 0.1 \times (-0.0192, 0.0329) = (0.1019, -0.1033).$$

7. Conclusion

After a single gradient-descent step:

- The loss decreased from approximately 0.1155 towards zero.
- All weights and biases were adjusted in the direction that reduces L .
- Repeating this process – feedforward, compute loss, backpropagate, update weights – over many iterations (epochs) across the entire training dataset causes the network to converge towards a set of weights that minimize the loss, thereby improving its predictive accuracy.

In practice, this procedure is applied to large datasets using efficient matrix operations and optimizers such as stochastic gradient descent (SGD) or Adam, but the underlying mathematics illustrated here remains the same.

References

References

- [1] Werbos, P. J. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University.
Relevance: The earliest application of reverse-mode automatic differentiation to neural networks. Werbos showed how the chain rule could be applied systematically to compute gradients of a loss function with respect to every weight in a multilayer network — the mathematical foundation of the backpropagation algorithm demonstrated in this document.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, **323**(6088), 533–536. <https://doi.org/10.1038/323533a0>
Relevance: The landmark paper that popularised backpropagation for training multilayer networks. It formalised the weight-update rule $w \leftarrow w - \eta \partial L / \partial w$ used in Section 6, and demonstrated that hidden units trained with this rule learn internal representations that capture structure in the data.
- [3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, **86**(11), 2278–2324. <https://doi.org/10.1109/5.726791>
Relevance: A comprehensive treatment of gradient-based training in fully connected and convolutional networks. The paper details the use of the sigmoid activation function and sum-of-squares loss — exactly the loss function and activation function used in Sections 3–5 of this worked example.
- [4] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. ISBN: 978-0-262-03561-3. <http://www.deeplearningbook.org>
Relevance: The standard graduate-level textbook reference for neural network theory. Chapter 6 covers feedforward networks and the backpropagation algorithm in the same matrix-vector notation used throughout this document, and provides a detailed derivation of the chain-rule expressions for δ_2 and δ_1 computed in Section 5.